

One Timetable

A scheduling and room-booking system for IIT Delhi Abu Dhabi

Evan Johan Tobias

Draft 1 — 11 September 2026

Contents

Summary	2
Part 1 — The case	3
Part 2 — What exists, and what we should do about it	5
Part 3 — Scope	7
Part 4 — The domain model	8
Part 5 — The five flows	9
Part 6 — How it should look	11
Part 7 — The wider system	15
Part 8 — Failure modes	16
Part 9 — Architecture	18
Part 10 — Rollout	20
Part 11 — Decisions management must make	21
Part 12 — What already exists, and how to check it	21
Appendix A — Sources	22
Appendix B — Reproducing the findings	23
Technical appendices	24
Appendix C — Schema	24
Appendix D — The validation catalogue	28
Appendix E — API surface	29
Appendix F — State machines	30
Appendix G — Screen specifications	31
Appendix H — Emitting iCalendar	32
Appendix I — Permissions	33
Appendix J — Testing	33
Appendix K — Effort	34

Summary

The ask. Approval to build a scheduling and room-booking system for the campus, starting with a two-to-three week read-only phase that costs the office nothing and delivers something it has already asked for.

Why. The timetable is maintained in five Excel workbooks, exported to eleven PDFs, and published with no step between that checks anything. Every published sheet and every workbook, read mechanically on 10–11 September 2026:

Rooms named inconsistently across sheets	6
Rooms double-booked across cohorts	1
Blocks whose end time precedes their start	2
Held sheets out of date against what was published	5 of 11
A room name contradicted by its own door plate	1

Five of those six fall to a one-week validation script, and that should be done whatever else is decided (§2.2). The case for a system is what remains after they are gone:

- Six revisions in the first three weeks of term. Each one silently invalidated every copy anyone held. Nothing announces a reissue.
- No room-wise view exists — the academic office asked for one on 11 September, and it cannot be produced from eleven PDFs by any amount of checking.
- Booking is an email thread with no record of who asked or who approved.
- “Is M4-0-019 free on Thursday at three?” cannot be answered without opening eleven files.

The defects are a symptom. There is no system of record — eleven documents are authoritative about overlapping facts and nothing reconciles them. That is a structural property, not a checking one, and it is the only thing here that cannot be scripted around or fixed by being more careful.

What is proposed. One authoritative schedule; room booking with approval; publishing to web, calendar feeds and PDF; conflict prevention enforced by the database rather than by care; change notification; utilisation reporting. Student registration and add/drop are deliberately out of scope for v1.

Build, and probably build less than first proposed. Commercial products (25Live, CELCAT, Syllabus Plus) are scoped for institutions with tens of thousands of students; UniTime is free but is a timetabling *solver* for a problem we do not have. But the campus already pays for Exchange room mailboxes, which can carry ad-hoc booking, and the strongest version of this proposal uses them and builds only the academic layer on top (§2.3). That also removes most of the “who maintains it” risk. IT should answer that question before Phase 2 is designed.

If the answer is “not now”: §2.2 rung 1 is a one-week validation script that catches most of the defects above and should be done regardless.

Cost. One developer, part-time. Sixteen weeks to a system that publishes, validates and books, in five phases each of which is useful on its own (Part 10). The first phase is three weeks and much of it already exists and runs.

What management must decide before Phase 2 (Part 11). Where it runs and who maintains it after I graduate; whether it becomes the system of record or stays a publisher; who approves bookings for each room and who deputises; and whether teaching may evict an approved booking.

The honest risk is not the code. It is that nobody is named to own this before the institution starts depending on it.

Part 1 — The case

1.1 What the current process is

The schedule lives in five Excel workbooks maintained in the academic office. Those are exported to PDF through Acrobat PDFMaker and uploaded to iitdabudhabi.ac.ae/timetable as eleven separate files, one per cohort. Everything downstream — students, faculty, room users — reads the PDFs.

There is no step between “someone edits a cell” and “eleven PDFs are correct and everyone knows”.

1.2 Defects this produces, measured

Running every published sheet and every workbook through a reader built for this purpose, on 10–11 September 2026:

Finding	Count	Example
Rooms called different names on different sheets	6	M4-0-019 is “Classroom 3”, “Classroom 5” and “Classroom 8”
Rooms double-booked across cohorts	1	Wed 17:00 M4-0-019: Masters AAIL7024 vs Y2 EEN AMLL1001
Blocks whose end time precedes their start	2	Y1 ELE and Y1 EEN, Monday AMLL1001: “15:30 to 14:20”
One class described two ways	1	AHUL261 Mon 09:00 — “Tutorial” on the CSE sheet, unlabelled on EEN
Room name contradicted by its own door plate	1	Y1 CHE calls M4-0-011 “Lecture Hall”; the plate says Classroom 3

Two of these compound: the Y2 EEN row in the double-booking prints code M4-0-019 beside the name “Classroom 8”, and Classroom 8 is M4-1-011 everywhere else. Either the code or the name is a typo, and which one it is decides whether the clash is real. Nobody can answer that from the documents.

1.3 What a one-week script would fix, and what it would not

Before arguing for a system, the honest bridge — because §2.2 puts a one-week validation script on the table and most of the table above falls to it.

Finding	Fixed by validation alone?
Impossible times (“15:30 to 14:20”)	Yes
Room double-booked across cohorts	Yes
Rooms named inconsistently	Yes, as a warning
One class described two ways	Yes
Name contradicted by its door plate	Yes, given the plate is recorded
Held sheets out of date	No

Five of six defects fall to a week of work with no server. A proposal that does not say so is not being straight, and that rung should be taken whatever else is decided.

So the case for a system does not rest on the defect list. It rests on what is left after the defect list is gone.

Two findings that only a cross-cutting view produces, both surfaced while checking this document's own numbers on 11 September:

One cohort has no published timetable at all. The workbooks contain twelve cohorts. The website publishes eleven. 26A1AIBMSEM1 — a Masters AI intake, Sem 1, four scheduled blocks — is in the source workbook and has no PDF. Its students cannot download a timetable because none was ever exported.

A cohort is labelled with the wrong semester. The Year 2 workbook contains 25A1ChEBSEM1. Year 2 Chemical is in Sem 3, and its siblings in the same file are 25A1CSEBSEM3 and 25A1EENBSEM3. The sheet name says Sem 1.

Neither is visible from inside any single document. Both are obvious the moment all twelve are in one place, which is the argument of §1.5 in miniature.

1.4 What remains, and it is the larger half

Distribution. Six revisions in the first three weeks of term. Revision dates carried by sheets actually issued: 23 August, 27 August, 7, 8, 9 and 10 September. Term began on 20 August. Each reissue means eleven PDFs re-exported and re-uploaded, and each one silently invalidates every copy anyone is holding — the phone screenshot, the printout on a desk, the file in a WhatsApp group. A validation script runs on the author's machine. It cannot reach any of those.

That is why 5 of 11 held copies were out of date on 10 September, and why two of them had been stale for thirteen days without anything indicating it.

There is no room-wise view, and the office has asked for one. On 11 September the academic office sent the source workbooks and asked, in return, for room-wise schedules. That view cannot be produced from eleven PDFs by any amount of validation, because the question crosses every file at once. It is a pivot, and a pivot needs the data in one place.

Booking is an email thread. There is no record of who asked, who approved, or on what basis. Nothing prevents a room being promised twice, and nothing tells the timetable that a room is spoken for. A script cannot introduce a record where none exists.

“Is M4-0-019 free on Thursday at three?” cannot be answered. Not by anyone, without opening eleven files and reading them by eye. This is the single most common question about a room and the current process has no mechanism for it at all.

1.5 The underlying problem

The defects are a symptom. There is no system of record.

Eleven documents are authoritative about overlapping facts, and nothing reconciles them. The same room has three names because three files each get to have an opinion. Two cohorts can hold the same room because no file can see the other. Copies go stale because a file is a snapshot and nothing announces that a newer one exists.

Validation makes each document internally consistent. It cannot make eleven documents into one truth, because that is a structural property and not a checking one.

That is the thing worth building, and it is the only thing in this document that cannot be bought cheaply, scripted around, or done by being more careful.

1.6 What this costs

- A student walks to the wrong room, because the name on the sheet is not the name on the door.
- Two classes arrive at the same room, and the more junior group leaves.
- A change is published and reaches nobody until someone re-downloads.
- The office answers room questions by reading eleven files.
- Six times in three weeks, everyone's copy became wrong and nobody was told.

None of this is carelessness. It is what happens when eleven documents are the system of record and a spreadsheet is the only integrity check.

What I have not measured: how much time the office actually spends on this. Everything above is measured from the documents; the staff-time cost is inferred and should be asked rather than assumed. If the honest answer is “very little”, the case for Phases 2 onward weakens and this document should say so.

Part 2 — What exists, and what we should do about it

2.1 The market

System	Vendor	Shape	Fit here
25Live	CollegeNET	Cloud. Academic + event + resource scheduling, approval workflow, conflict checking. Market leader in North American HE.	Model is right, scale and cost are not
Syllabus Plus / Granular	Scientia	Capacity-aware section placement, automated timetabling	Enterprise; built for thousands of sections
CELCAT	CELCAT	Timetabling + room booking + SIS integration, strong UK presence	Same
Untis	Untis GmbH	Schools, mostly Europe, integrated SIS	Wrong segment
UniTime	Apereo Foundation	Open source, Java. Course + exam timetabling, student scheduling, room sharing, distributed departmental editing	Closest technical match; heavy

2.2 The ladder, cheapest first

A proposal that jumps straight to “build it” invites the obvious reply. So here is every rung, honestly costed, including the ones that make this document unnecessary.

Rung 1 — Validate the workbook before export. About one week. A script the office runs on the workbook: the rules in Appendix D, no database, no server, no login. This alone catches most of §1.2 — the impossible times, the double-booking, the naming conflicts, the credit mismatches.

It does not fix distribution: sheets still go stale, nothing notifies anyone, there is still no room-wise view and no booking. But it is a week, and if the answer to this proposal is “not now”, this is the part to do anyway.

Rung 2 — One shared workbook instead of five. About zero weeks. Consolidating the workbooks removes some duplication by hand. It does not survive contact with eleven cohorts and four authors, and it makes the single-file conflict problem worse rather than better. Listed for completeness; not recommended.

Rung 3 — Use what the institution already pays for. See §2.3, which is the serious alternative and was missing from drafts 1 to 6 of this document.

Rung 4 — Buy a product. 25Live, CELCAT, Syllabus Plus. Priced and scoped for institutions with tens of thousands of students and a dedicated scheduling office. IITD-AD has eleven cohorts and one person maintaining the workbooks.

A gap I have not closed: I have not obtained pricing for any commercial product. The judgement that they are disproportionate is based on their scale and target market, not on a quote. If management wants the comparison made properly, a quote should be requested — and if one of them turns out to be affordable at this size, buying beats building, and this document should lose.

Rung 5 — Adopt UniTime. Free, open source, Apereo-governed, and technically the closest match. But it is a large Java/Hibernate application built around an automatic timetabling *solver*, for distributed departmental scheduling at a research university. We do not have the problem it solves. Our schedule is already built, and built well, by a person who knows what she is doing. What fails is everything after that. Adopting it means operating a system far larger than the problem, and the institution would still need someone to run it — which is the same ownership question as building, with less control over the outcome.

Rung 6 — Build a focused tool. The recommendation, for the reasons the rest of this document gives. Copy 25Live's domain model — request, approve, detect, publish — rather than its software.

2.3 The option already paid for: Microsoft 365 room mailboxes

The campus runs Outlook and Teams. Exchange includes room resource mailboxes, which already do a meaningful part of Part 5.3: a room is a mailbox, meeting requests to it are auto-accepted or declined by the Resource Booking Assistant, conflicts are refused, and a delegate can be required to approve. It is licensed, maintained by someone else, needs no new host, and faculty already know it.

Leaving this out of drafts 1 to 6 was the biggest omission in the document, because it is the first thing any competent IT person in the room will raise.

What it genuinely gives:

- Room availability inside Outlook and Teams, where people already are
- Conflict rejection without anyone writing it
- Delegate approval with `AllBookInPolicy` / `AllRequestInPolicy`
- No server to host, patch, or own after I graduate — which answers the single largest risk in Part 11

What it cannot do, and these are not small:

- It has no idea what a cohort, a course, a group or a credit is. It cannot validate that ACOL351 has the tutorial hours its L-T-P-C says, because it has never heard of L-T-P-C. Every defect in §1.2 except the double-booking survives untouched.
- No import, no revision, no diff. The workbook problem — five workbooks, eleven exports, five stale copies — is entirely unaddressed.
- No per-cohort publishing. Students do not have a “Y3 CSE” mailbox.
- Recurring series are all-or-nothing by default. Exchange checks every occurrence and declines the *entire series* if one conflicts, tunable only through `ConflictPercentageAllowed` and `MaximumConflictInstances`. A term-long teaching pattern that clashes once in week nine is refused outright, which is precisely the case §8.1 exists to handle gracefully.
- No precedence. Exchange cannot express “teaching outranks a society booking”; whoever asked first wins.

So the honest recommendation is a hybrid, and it is better than what drafts 1 to 6 proposed:

Exchange room mailboxes	the booking substrate for ad-hoc requests
	– availability, approval, invitations, reminders, and no host to own

This system	the academic layer
	– import and validate the term
	– cohorts, groups, credits, revisions
	– publish per-cohort views and feeds
	– push teaching occupancy INTO the room mailboxes so Outlook shows the real picture

Teaching is published to the room mailboxes as busy time. Ad-hoc booking then happens in Outlook, against a calendar that finally knows when classes are — which today it does not, which is why room conflicts are found by walking into them.

The trade, stated plainly: the exclusion constraint in Appendix C stops being the single source of truth for ad-hoc bookings, because Exchange owns those. The system keeps it for teaching, where it matters most and where precedence lives. Two systems holding room state is a real cost, and it buys the removal of the largest risk in the proposal.

This needs IT's answer before Phase 2 is designed, and it is now question 7 in Part 11.

2.4 The objection to me building it

It should be in the document rather than left for the meeting.

A student is proposing to build infrastructure the institution would come to depend on, and that student graduates.

That is correct, and it is the strongest argument against this proposal. The answers that are honest rather than reassuring:

- The phasing is the mitigation. Phase 0 is read-only and disposable — if it is abandoned, the office has lost nothing and gained a room-wise view. Nothing becomes load-bearing until Phase 2, which is why Part 11's ownership question is gated to before Phase 2 rather than before Phase 0.
- Full Excel export, always. The institution's data leaves whenever it wants, in the format it already works in. There is no lock-in to inherit.
- §2.3's hybrid halves the exposure. If ad-hoc booking lives in Exchange, the part that must be maintained forever is the academic layer, not the booking infrastructure.
- Boring on purpose. Postgres, one application, server-rendered pages. No framework that needs a specialist. A competent developer should be able to pick it up from the schema and Appendix D.
- What I cannot promise is that someone will want to maintain it. That is a staffing decision, not an engineering one, and pretending otherwise would be the dishonest part of this document.

2.3 The one thing worth taking from the standards, not the products

Emit iCalendar (RFC 5545). Do not implement CalDAV. A read-only .ics feed over HTTPS gets roughly 95% of interoperability value for a fraction of the work; CalDAV (RFC 4791 plus RFC 6638 scheduling) is what a full two-way calendar *server* implements and is enormous overkill for publishing.

And a finding that changes the design: Google Calendar refreshes subscribed ICS feeds every 12 to 24 hours, is not configurable, has no force-refresh, and ignores both REFRESH-INTERVAL (RFC 7986) and Microsoft's X-PUBLISHED-TTL. Apple Calendar and Thunderbird poll every 5–15 minutes; Outlook web about 3 hours. Google is both the slowest and the most used.

An ICS feed can never be the notification channel. A room change posted at 09:00 does not reach a Google subscriber until the next day. Split the responsibilities on day one: the feed is “what's on this month”, the web app is the source of truth for “is this still on and where”, and same-day changes go out through a separate push path.

Part 3 — Scope

3.1 In scope for v1

1. One authoritative schedule for every cohort, every term.
2. Room booking: request, approve, and see availability.
3. Publishing: web views, per-cohort and per-room, plus ICS feeds.
4. Conflict prevention at write time, not discovery after the fact.
5. Change notification with a human-readable diff.
6. Utilisation reporting — frequency rate in v1; occupancy needs enrolment data that does not exist yet (§5.5).

3.2 Explicitly out of scope for v1

- Student course registration and add/drop. Real, wanted, and a different system. Staged as v3 in Part 10.
- Automatic timetable generation. Constraint solvers are the most expensive part of every product above and solve a problem the office does not have.
- Exam scheduling. Separate cycle, separate constraints.
- Attendance. Different system entirely.

Saying no to these in v1 is what makes v1 shippable.

Part 4 — The domain model

4.1 Entities

Grouped by what they are, because the grouping is itself the argument: three layers that change at completely different rates.

The estate. Changes once a decade.

Building	M1 M2 M3 M4
Room	code (identity), capacity, kind, building, floor, approver, deputy
Room name claim	a name ONE SOURCE gives a room, with where it came from

The calendar and curriculum. Changes once a term.

Term	2026–27 Sem 1, start, end
No-class day	holidays, mid-sem break, exam weeks
Course	code (identity), title, L–T–P–C, department
Cohort	24A1CSEBSEM5 → "Y3 CSE Sem 5", headcount, number of groups
Section	a course taught to a cohort this term, with an instructor, an enrolment kind, and registered heads
Section shares	the other cohorts attending that same section

What actually happens, and when. Changes weekly.

Meeting	the editable pattern: room, weekday, time, groups, from/until
Occupancy	one instance of a meeting, in a room, at a time
Booking	a request for non-teaching use, with a status and a decision
Revision	a published state of a term, with a diff against the last one
Person	staff, faculty, student; role-scoped
Audit	who changed what, when

Meeting and Occupancy are two entities on purpose, and this is the single most important line in the model. A weekly class is one editable fact and sixteen things that occupy a room. The pattern is what a human edits; the instances are what the database polices. Collapse them and you get either a system that cannot detect a clash or one where moving a class means rewriting sixteen rows.

4.2 The rules that make it hold together

1. A room's code is its identity. Every name is an attribute. M4–0–019 is the room. "Classroom 5" is what the door says. "Classroom 3" is what one sheet claims and "Classroom 8" is what another does. All are stored as *claims with a source*, none is the key, and the disagreement stays visible rather than being silently resolved by whichever file was read last. This one rule retires all six naming defects in §1.2.

2. Nothing references a course except by code. Not by title, not by short name. A course legitimately has several names — the official title, the everyday name, the abbreviation on a pill — and each is a field on one object.

3. A meeting names its groups explicitly. {} means the whole cohort. {1} means group 1 only. This is exactly what the workbooks encode as merged cell spans and exactly what the PDF export destroys. Year 1 runs four groups with A/B/C sub-splits, so “two groups” must never be assumed anywhere.
4. A class taught once to several cohorts is one thing. Not one per cohort. In the current term, 248 cohort entries are 160 real bookings; without this the other 88 read as double-bookings. Section shares carries it.
5. Teaching and bookings share a table but not a priority. One occupancy table, so they cannot overlap unnoticed. An explicit precedence (§5.3), so a society’s booking cannot block the institution from publishing its own timetable. Precedence decides who is *asked* to move, never who is moved silently.
6. Every published state is versioned. A revision has a stamp, an author, and a diff against its predecessor. “What changed between the 7th and the 8th” is a query, not an archaeology exercise — and a bad publish is undone by pointing the term at the previous revision rather than by restoring a backup.
7. Nothing is ever *just* deleted. Cancelled occupancy keeps its row and releases its slot through a partial index. An evicted booking records which revision displaced it. The audit log keeps the before and after. In a system whose entire purpose is settling disagreements about who had which room, destroying evidence is the one unrecoverable mistake.

4.3 What is deliberately NOT modelled in v1

Naming these is how scope stays honest, because each is a plausible next question and each is a term of work:

- Student enrolment as individuals. Sections carry a registered *count*, not a roster. A roster is add/drop, which is Part 10 Phase 5.
- Attendance. Different system, different consent question.
- Instructor workload and contracts. HR’s, not scheduling’s.
- Physical seating layout. Capacity is a number, not a seat map. Exams will eventually want the seat map (§7.6) and that is when to model it.

Part 5 — The five flows

5.1 Build a term

The office keeps working in Excel. That is not a compromise, it is correct: the workbook is where the thinking happens, and forcing a grid editor on someone who is fluent in Excel buys nothing.

- 1 scheduler uploads the term workbook
- 2 system reads it: cohorts, meetings, groups from merge spans, the L-T-P-C table
- 3 system validates BEFORE accepting:
 - every course in the grid appears in the credit table, and vice versa
 - contact hours reconcile against L-T-P-C, per group
 - no end time precedes its start
 - every room code exists
 - no two meetings occupy one room at one time
- 4 clean → staged, with a diff against the current published term
not clean → rejected, with the exact cells listed. Nothing is written.
- 5 scheduler reviews the diff and publishes

Step 3 is the whole value. Both “15:30 to 14:20” errors and the double-booking would have been refused at upload.

5.2 Publish

One publish writes every view at once:

- per-cohort week (web)
- per-room week (web) — *what Priya asked for on 11 September, and does not exist today*
- per-instructor week (web)
- ICS feed per cohort, per room, per instructor

- a PDF export per cohort, byte-for-byte the layout people already know

The PDF export matters politically. It means adopting the system costs nobody their familiar artefact, and it removes the argument “but we need the PDF”.

5.3 Book a room

requester picks a room and a time

- > system shows availability live, from the same table teaching uses
- > submit creates a request in PENDING
- > routing rule sends it to the approver for that room
- > approver sees the request beside that room's day
- > APPROVE writes a booking; the exclusion constraint refuses an overlap
- > requester is notified; the room's views and feeds update

A booking and a class are the same kind of row. If they are separate tables, they will overlap eventually.

But they are not the same priority, and draft 1 of this document got that wrong. Treating them as equals means the exclusion constraint refuses a *publish* that collides with an approved booking — so a student society's room request can block the institution from publishing its own timetable. That is obviously the wrong answer, and it is a policy decision with a technical consequence, so it has to be stated rather than inherited from whichever row happened to be written first.

Precedence, highest first:

- | | |
|----------------------|---|
| 1 exam | fixed by a separate calendar, immovable |
| 2 teaching | the institution's core function |
| 3 maintenance hold | estates has taken the room out of service |
| 4 approved booking | someone was told they had it |
| 5 provisional / hold | not yet promised to anyone |

Precedence never silently overwrites. It decides *who is asked to move*, not who is moved without being told.

5.4 Change a published schedule

edit -> validate -> diff -> publish -> notify

The notification is a sentence, not a file:

Revision 6, 11 September 14:20. ACOL351's Wednesday tutorial moved from M4-0-019 to M4-0-011, same time. One other change: AHUL261's Monday tutorial is now 09:00, was 15:30. Nothing else changed.

The office already writes these by hand in email. Generate them: sets in, sets out, sets moved.

5.5 Report utilisation

The standard measures, which are what management actually asks for:

- Frequency rate — share of available hours a room is timetabled
- Occupancy rate — expected heads against room capacity while in use
- Utilisation — the two multiplied; the headline estate figure
- Show rate — expected against actually present, where attendance data exists

Occupancy rate is the one that is easy to get wrong, and v1 of this document did. It needs expected attendance per *meeting*, not cohort size. A group-1 tutorial is half a cohort. An elective marked “Applicable if registered” might be a fifth of it. A shared HUL lecture is two cohorts at once. Divide room capacity into cohort headcount and every elective and every group-split tutorial reports as a near-empty room, which is exactly the finding that would get an estates decision wrong. The expected column on *meeting* exists for this and must be populated before any occupancy figure is published.

Frequency rate needs none of that and should ship first. It answers “do we need more rooms” and “which rooms are never used” from the timetable alone, and both are live questions on a campus this size. Occupancy and utilisation wait for enrolment data; publishing them early with cohort headcount as a stand-in would be worse than not publishing them.

Part 6 — How it should look

6.1 Principles

Show state in form, not only in colour. Provisional, confirmed, cancelled and pending must each be distinguishable without relying on hue.

Colour encodes kind, not identity. Lecture, tutorial, lab, reserved. Colouring by course produces a rainbow that means nothing at a glance and breaks with more than about eight courses.

Never render an unverified value as though it were verified. If a room name is disputed across sources, the view says so rather than picking silently.

6.2 The views

Cohort week — horizontal. Days down the side, time across the top, matching the orientation of the printed sheet people already know. Group badges on any block that does not apply to the whole cohort. This is the default landing view.

Room day — the booking surface. One room, one day, time across. Existing teaching in solid blocks, bookings in a lighter fill, free time genuinely empty and clickable. Dragging across free time opens a request pre-filled.

Room week grid — the office view. Rooms down the side, days across, each cell a compressed bar of what is in it. This is the one that makes a clash visible without anyone looking for it.

Availability finder. “I need a room for 90 minutes on Tuesday afternoon that seats 40.” Returns rooms, ranked by how well the capacity fits, because sending a seminar of 12 into a 120-seat lecture hall is a real cost.

Utilisation heatmap. Rooms against hours, colour by frequency. Dead rooms and pressure points both become obvious in one screen.

Mobile. Single column, and the top of the screen answers one question: what is next, where, and how long until it starts.

6.3 Showing a room, five ways

A room is not one thing on a screen. It is a row in a list, a shape on a plan, a volume in a building, a sign on a door, and a line on a phone held while walking. Each answers a different question and they are not substitutes.

As a row — “what is in it this week”. The dense view. Room down the side, time across, every booking a bar. This is the office’s working surface and the only one that shows a clash without anyone looking for it.

As a shape on a floor plan — “which door”. The plan of one level, rooms filled by live status: in use, free, free-but-booked-soon, unbookable. A person who does not know the building does not think in room codes; they think “the corner one past the stairs”. LinkCS already carries redrawn floor plans for M3 and M4 ground and level 1, with room polygons keyed to codes. This is the largest existing asset and it is currently attached to nothing.

As a volume — “which building, which floor”. The 3D view, already built. Its job is orientation, not detail: a new-comer seeing M3 and M4 side by side with the busy floors lit. It should never be the primary booking surface — 3D is worse than a plan for picking a target — but it is the best answer to “I have never been in this building”.

As a sign on the door — “am I in the right place, now”. A small always-on display, or at minimum a printed QR, at each teaching room. Shows the current class, the next one, and the time until the room frees. This is where the institution’s naming problem becomes visible to everyone: the sign and the door plate must agree, and today across eleven sheets six rooms do not.

In the hand — “where am I going and can I get there”. Phone. Room, floor, walking direction from where the person is standing, and the time budget. This is the view that should know the campus bus runs, because a 09:00 in M3 is a different proposition if the bus arrives at 08:55.

Every one of these reads the same occupancy table. The moment a floor plan has its own copy of “which rooms are busy”, the copies drift, and this whole document exists because of drift between copies.

6.4 Status, precisely

Five states, each needing a distinct form and not only a colour:

State	Means	Form
Free	Nothing scheduled, bookable now	Empty, outlined, clickable
In use	Teaching or an approved booking	Solid fill, with what and until when
Reserved soon	Free now, booked within 30 min	Outlined with a countdown
Pending	Requested, not yet approved	Hatched
Unavailable	Maintenance, exam hold, not bookable	Diagonal grey, no click target

“Free” must mean *free*, not “nothing on the sheet I happened to read”. A room the system has no data for is unknown, not free, and says so. That distinction is the honest version of the Free Rooms page that exists today.

6.6 Booking, second by second

Every other flow in this document is described from the system’s side. This one is described from the user’s, because the gap between them is where a booking system is actually judged.

- 0s room day view. Free time is empty, outlined, and the only clickable thing on the row. Nothing else invites a click.
- 1s drag across 14:00–15:30. The range fills as it is dragged and shows “1h 30m” while dragging, not after.
- 2s a panel opens, already carrying room, date and time. The only empty fields are purpose and headcount. The submit button says “Request room” – never “Submit”.
- 4s submit. The slot immediately shows as PENDING, hatched, with your name on it.

The slot does not turn green. It has not been approved. An interface that shows a request as though it were a booking teaches people to turn up to rooms they do not have.

And the race, which §8.4 handles in the database and the interface must handle too. Between the page loading and the request submitting, someone else can take the slot. Then:

That 14:00 slot was approved for someone else two minutes ago. Your request was not lost — here it is, and these three rooms are free at the same time.

The request is retained and re-targetable. Making the user retype a form they already filled is how a system earns the reputation the email thread had.

The approver’s side is a queue, not a calendar. Oldest first, each row carrying who, what for, how many people, the room, and the time — and, critically, what else is in that room that day, so approving does not require opening another view. Two buttons. A refusal opens a reason field, because §8.3 makes the reason mandatory.

6.7 When there is nothing to show, or the wrong thing

Part 8 gives the system ten failure modes. Drafts 1 to 4 gave the interface none. Every screen needs these seven states specified, or they get invented under deadline:

State	When	What it says
Loading	First paint, data not back	Skeleton of the actual grid, never a spinner on blank. The shape should not jump when data lands.
Empty	Term exists, nothing scheduled	“No classes published for Sem 5 yet.” A scheduler also sees a link to upload. Never an empty grid, which reads as a free week.
Unknown	No data for this room at all	“No schedule held for M2-2-031.” Not “free”. The difference between “nothing is booked” and “we do not know” is the difference between a correct answer and a wrong one.
Stale	Last successful fetch is old	The view greys and says how old it is. On a wall display this matters more than anywhere: a frozen board looks authoritative and lies.
Offline	Request failed, device has no network	Last known data stays on screen, marked with its age, and writes are refused with “you are offline” rather than queued silently.
Denied	Authenticated, not permitted	Say which role is needed and who grants it. “Forbidden” tells a person nothing they can act on.
Error	Anything else	What failed, what to try, and a reference to quote. No apology, no “something went wrong”.

The stale state already exists and is already right. The campus wall board in LinkCS greys itself and prints “*not updating - reload the page*” if its clock has not advanced for two minutes, on the reasoning that a board showing an hour-old departure with full confidence is worse than a blank one. That rule transfers directly, and it is the single most important state on this list, because the wall display is the one screen nobody is watching.

Bad signal is the normal case, not the edge case. M3’s labs are interior rooms. The phone view must render from cache and say how old the cache is, and it must never let someone submit a booking it cannot confirm.

6.8 Accessibility, specified

The institution has already committed to this physically — every door plate carries Braille. A digital system that does not match that is a step backwards, so this section is requirements, not aspiration.

A timetable grid is one of the hardest things to make accessible, because a two-dimensional arrangement of absolutely-positioned blocks is close to meaningless to a screen reader. Position on a grid is not information a reader can convey.

- Every grid ships with a list view of the same data, reachable by a control and by keyboard, ordered by day then time. Not a lesser fallback — the same content, and for many people the better one.
- The grid is keyboard-operable: arrow between blocks, Enter to open, Escape to close. A booking must be completable without a pointer, since dragging a time range is exactly the interaction a pointer-free user cannot perform. The panel therefore also takes typed start and end times.

- Colour is never the sole carrier of status. Already stated in §6.1 and §6.4; the practical test is that the five states in §6.4 must be distinguishable in greyscale, which is also the photocopier test and the cheap-projector test.
- Contrast meets WCAG 2.2 AA: 4.5:1 for text, 3:1 for the boundary of any block whose fill carries meaning. Measured, not assumed — see the table below.
- Focus is always visible, and never removed for looking untidy.
- Motion respects **prefers-reduced-motion**. Nothing in a timetable needs to move.
- Arabic is right-to-left. Room labels are bilingual per §6.5, and if an Arabic interface is ever offered, the grid's time axis reverses with it. Worth deciding now whether that is in scope, because retrofitting direction is expensive and bolting it on late is how it gets done badly.
- Touch targets are at least 44px. The door and phone cases are one-handed, often while walking.

Measured contrast Every pair the interface actually uses, computed rather than eyeballed:

Foreground	Background	Ratio	Verdict
Ink #191919	White	17.58:1	passes
Muted #475569	White	7.58:1	passes
Muted #475569	Ground #F7F7F7	7.07:1	passes
Crimson #A41E22	White	7.53:1	passes
Crimson #A41E22	Ground	7.03:1	passes
Crimson #A41E22	Tint #FCE6E6	6.31:1	passes
Deep #690F12	Tint	10.49:1	passes
OK #2E7D5B	White	5.00:1	passes
Critical #B3261E	White	6.54:1	passes
Warning #96650C	White	5.04:1	passes

This table changed the palette, which is the point of measuring. Draft 4 specified warning as #B4770F, an amber that reads fine and scores 3.76:1 on white — below AA for normal text. It is now #96650C. Draft 4 also asserted that crimson fails as small text on the tint; it does not, it scores 6.31:1. One claim was a real defect and the other was confident and wrong, and only arithmetic told them apart.

Times are 24-hour throughout. The sheets are 24-hour, the institution is bilingual, and 15:30 is unambiguous in every language. The existing student site uses 12-hour, and that inconsistency should be settled deliberately rather than inherited.

6.5 Theming

The institution's own palette, taken from `iitdabudhabi.ac.ae`:

Primary	#A41E22	IITD crimson, the dominant brand colour on their site
Deep	#690F12	headers, hover states
Tint	#FCE6E6	selected rows, soft emphasis
Ink	#191919	primary text
Ink secondary	#2C2A2E	headings
Muted	#475569	labels, metadata
Rule	#E2E8F0	borders, grid lines
Ground	#F7F7F7	page background
Surface	#FFFFFF	cards, panels

Semantic colours are deliberately not crimson, so that “needs attention” never competes with brand:

OK	#2E7D5B	
Warning	#96650C	
Critical	#B3261E	(distinct from brand crimson, used only for real errors)

Typography. Their site uses Inter and Matter. Inter is freely available and should carry the interface. Matter is a commercial licence — if the institution holds one, use it for headings; if not, Inter alone is correct and consistent.

A monospaced face for room codes, times and L-T-P-C values. These are data, they line up in columns, and M4-0-019 in proportional type is harder to scan.

Bilingual. Door plates are Arabic and English. Room labels in the interface should carry both.



Part 7 — The wider system

Beyond v1, in rough order of value per unit of work. Nothing here is required for the first four phases to be worth doing, and none of it should be built before the schedule is trustworthy.

7.1 Wayfinding

Room code to a route: building, entrance, floor, turn. The floor plans exist and carry room polygons; the missing piece is a walkable graph — corridors, doors, stairs, lifts — over the same geometry.

Step-free routing is not a nice-to-have. The door plates carry Braille, so the institution has already committed to physical accessibility; a wayfinding feature that routes everyone up stairs undoes that commitment digitally.

7.2 Door signage

E-paper or small panels outside teaching rooms, driven by the same feed as everything else. Cheap version first: a printed QR per door linking to that room's live day. The QR costs nothing, needs no hardware budget, and tests whether anyone actually wants it before panels are purchased.

7.3 Equipment as a bookable attribute

Rooms differ by what is in them — projector, whiteboard, lab benches, power at every seat, a camera for hybrid teaching. Model equipment as room attributes and the availability finder answers “a room for 40 with a camera”, which is the query people actually have.

7.4 Maintenance and cleaning holds

Estates needs to take a room out of service. Today that is an email. As a hold row it is enforced by the same constraint that stops double-booking, and it appears in every view automatically.

7.5 Events, with RSVP

Club events, talks, workshops. A booking with a public face: capacity, a registration list, a reminder. The Coding Club's quiz night is a booking, a capacity, and a list of who is coming; today it is a room request plus a WhatsApp thread plus a spreadsheet.

7.6 Exams and invigilation

A separate cycle with harder constraints: no student in two exams at once, invigilator ratios, seat spacing that cuts effective capacity. Worth building only once teaching schedules are stable, and worth building because it is the single most painful scheduling task in any academic year.

7.7 Instructor preferences, collected before the term is built

The office currently discovers constraints by being told them, often late. A short form before timetabling opens — unavailable slots, preferred rooms, equipment needed — turns a class of late changes into an input.

This is the highest-leverage item in this list. Every late change costs a revision, a republish, and a room already booked by someone else.

7.8 Term-over-term planning

“Show me last year’s Sem 1 against this one.” Room pressure, course growth, which rooms were never used. Estates decisions are made on this comparison and it currently cannot be made at all.

7.9 Occupancy sensing

Last, deliberately. Sensors answer “was the booked room actually used”, which is a real question — but only once booking is reliable enough that the answer means something. Buying sensors before that measures noise.

7.10 Integration with the campus bus

The institution already runs a shuttle on a published schedule, and LinkCS already carries it, verified against the transport office’s own workbook. A student’s real question is not “when is my class” but “when do I need to leave”. Joining the two is a small piece of work against data that already exists on both sides, and no commercial product in Part 2 can do it, because none of them knows this campus has a bus.

Part 8 — Failure modes

Parts 5 and 7 describe what happens when things go right. A booking system is mostly the other case, and every row here is a decision someone has to make before the code is written.

8.1 Publishing over a booking

A scheduler publishes a revision. One of its classes lands on a room at a time someone already has an approved booking for.

What must not happen: the publish fails, and the timetable cannot go out because of a club meeting. Also must not happen: the booking silently vanishes and the requester finds out by turning up.

The flow:

stage the revision

- > compute evictions: approved bookings the new schedule would displace
- > if any, the scheduler sees each one: who booked it, what for, when
- > publish requires explicit acknowledgement of each eviction
- > inside ONE transaction:
 1. cancel the evicted bookings, with a reason and the revision number
 2. insert the new occupancy rows
 (order matters: the partial index only releases the slot once the booking is cancelled, so a reversed order fails on its own constraint)
- > each evicted requester is notified, with the reason and the three nearest alternative rooms at the same time

The alternatives matter. “Your booking was cancelled” is an email that generates a reply; “your booking was cancelled, here are three rooms free at that hour” is one that does not.

8.2 Rolling back a revision that bookings were made against

Between publishing revision 6 and discovering it was wrong, people booked rooms around it. Rolling back to revision 5 may re-open slots those bookings now sit in, or may collide with them.

Rule: rollback is a publish. It goes through §8.1 identically, including the eviction report. There is no separate rollback path, because a separate path is a path with fewer checks.

8.3 The approver is unavailable

Requests age silently while someone is on leave. This is the most common way a booking system quietly stops being used.

- Every room has a named approver and a named deputy. Not a role, people.
- A request unanswered for 48 hours escalates to the deputy and appears on the scheduler's dashboard.
- Unanswered after 5 days it auto-refuses with that reason, so the requester gets an answer rather than silence. A refusal can be appealed; an unanswered request cannot.

8.4 Two people request the same slot at the same moment

Both see it free, both submit. Both requests are valid — pending requests do not reserve anything, deliberately, because a system where requesting holds a room gets used to hoard rooms.

The approval is what serialises. The first approval writes; the second fails on the exclusion constraint and the approver is told, in that moment, that the slot has just gone and which request took it.

8.5 The first import, before any room exists

A bootstrap problem draft 1 walked straight past. Validation rule B3 requires every room code to exist in the room table. The only source of room codes is the imports. Nothing can ever be imported.

Resolution: the first import of a term runs in discover mode. Unknown room codes are collected and presented to an admin as proposals — code, the names the sheets give it, which cohorts use it — to confirm, merge, or reject. After the first term, an unknown room code is an error, because by then it means a typo.

8.6 A booking beyond the end of term

Recurring bookings must be clamped to the term, or a society books a weekly slot that silently extends into a term whose timetable does not exist yet.

Rule: a recurring booking's `until_date` cannot exceed the term end. Wanting next term means asking again once next term is published, which is also when the room situation is actually known.

8.7 Time, stated explicitly rather than got away with

Asia/Dubai does not observe daylight saving. That is why a naive implementation would work here and break the moment anyone reuses it. Store local time with an explicit `tz`, and derive UTC instants; never store bare UTC and reconstruct local. If the term shifts, the *local* time is what was meant.

Related: the sheets print 12:00–14:00 as a lunch band and a few blocks run past 19:00. The teaching day is not 08:00–18:00 and validation rule B2 must not assume it is.

8.8 Concurrent edits to one meeting

Two schedulers open the same meeting. Last write wins is wrong; it loses an edit silently.

Every meeting carries a version. A save sends the version it was loaded at, and a stale version is refused with a diff of what changed underneath. This is cheap and it is the difference between “someone else changed this, here is what they did” and a half-applied timetable.

8.9 The import parses but is wrong

Every validation rule in Appendix D passes and the schedule is still wrong, because the workbook itself was wrong.

There is no technical answer to this. What the system owes is traceability: every meeting records the revision, the source file, and its hash. “Where did this Wednesday 15:30 come from” is answerable in one query, which is what turns an argument into a lookup.

8.10 Nobody uses it

The realistic failure. The office keeps using email because the new thing is one more place to check.

The mitigations are product decisions, not technical ones, and they are why the phasing in Part 9 is ordered the way it is:

- Phase 0 gives the office something it asked for (the room-wise view) before asking it to change anything.
- The PDF export means adoption costs nobody their familiar artefact.
- Full Excel export at any time means the institution is never trapped.
- Approvals arrive by email with approve and refuse links in the message, so an approver never has to visit the system to use it.

Part 9 — Architecture

7.1 Stack

Database	PostgreSQL
Backend	one application server; Python/FastAPI or Node/Fastify
Frontend	server-rendered pages plus light client JS
Auth	institutional SSO (Microsoft 365; campus already runs Outlook and Teams)
Hosting	on campus, on institution-owned infrastructure

Postgres is not interchangeable here. The core guarantee is a database constraint only it provides.

7.2 The constraint that does the real work

```
CREATE EXTENSION btree_gist;
```

```
CREATE TABLE occupancy (
  id          bigserial PRIMARY KEY,
  room_id     text       NOT NULL REFERENCES room(code),
  during      tstzrange  NOT NULL,
  kind        text       NOT NULL,          -- 'class' | 'booking' | 'hold'
  status      text       NOT NULL DEFAULT 'confirmed',
  EXCLUDE USING gist (room_id WITH =, during WITH &&)
  WHERE (status <> 'cancelled')
);
```

Two requests arriving in the same millisecond both pass an application-level check; the constraint refuses the second row. This is the difference between “we try not to double-book” and “we cannot double-book”. Making it partial (`WHERE status <> 'cancelled'`) matters, or cancelled events hold their slots forever.

Pieces the failure modes require

Part 8 added three moving parts the draft-1 architecture did not have. Naming them matters, because each is a thing that can be down.

A job runner. Escalation at 48 hours, auto-refuse at five days, nightly feed regeneration, backup. One scheduled worker, not a queue cluster — but something has to run on a timer, and “a cron entry calling a management command” is a legitimate answer at this scale as long as it is *written down* rather than discovered later.

A mail path. Approvals, eviction notices, escalations. Sent through the institution’s own mail, not a third-party service — these carry student names and room bookings, and routing them via an external provider is a data question nobody asked for. Approve and refuse links go in the message body, so an approver never has to visit the system to use it (§8.10).

Observability, at the level this actually needs. Not a dashboard stack. Three things: a health endpoint that checks the database, a log of every publish and every approval, and one alert — *the job runner has not run in 24 hours*. That single alert catches escalations silently stopping, which is the failure that would otherwise be discovered by a complaint.

Hosting, said plainly

One virtual machine on institution infrastructure, with Postgres on the same host. At eleven cohorts and about three hundred meetings, that is ample, and pretending otherwise would be architecture theatre.

The consequence, stated rather than hidden: one machine means one point of failure. If it is down, nobody can book or publish. Published views should be servable from static snapshots regenerated on each publish, so an outage degrades to “you cannot change anything” instead of “nobody can see their timetable” — which is the difference between an inconvenience and an incident.

Someone must patch it. That is part of Part 11’s ownership question and not a separate one.

Data, retention, and what the system knows about people

The system holds: names and institutional email for staff and faculty; cohort membership for students; and who booked which room for what. That last one is a record of people’s movements, and should be treated as such.

- Booking history is retained for the academic year, then reduced to counts for utilisation reporting with the requester removed.
- The audit log is retained for two years, because it exists to answer “who changed this” and a one-term window makes it useless.
- No student is individually identifiable in any published view.
- Full export belongs to the institution at any time.

Worth confirming against whatever policy the institution already has, rather than inventing one here.

7.3 Roles

Viewer	public. Published schedules, room availability. No login.
Student	their own cohort surfaced first; subscribe to feeds.
Faculty	their own teaching; request rooms.
Approver	approve or refuse requests for rooms they own.
Scheduler	upload a term, edit meetings, publish revisions.
Admin	rooms, users, terms, approval routing.

7.4 Operational requirements

- Every write is attributed and timestamped. “Who moved this and when” must always have an answer.
- Nightly database backup, restore tested at least once before go-live.
- Read-only mode that keeps published views serving if the admin side is down.
- Full export to Excel at any time. The institution must never be locked into this tool, and saying so in writing is what makes it adoptable.

Part 10 — Rollout

Each phase is independently useful. If the project stops after any one of them, what was delivered still stands on its own.

These estimates are revised upward from draft 1, because drafts 2 and 3 found work that was not in the plan: the pattern/occurrence split, room discovery mode, shared sections, and the whole of Part 8. An estimate that does not move when the design moves is not an estimate.

Phase 0 — Publish what already exists (3 weeks, was 2)

Read-only. Import the current workbooks, publish cohort views, room views, and ICS feeds. No booking, no editing, no login.

Now also includes, from later drafts: the meeting / occupancy split and its transactional regeneration (§Appendix C), room discovery mode so the first import is possible at all (§8.5), and `section_shares` so co-taught classes do not read as clashes (248 cohort entries are 160 bookings).

Delivers: the room-wise schedule the office has already asked for; one URL that is always current; the end of the eleven-stale-PDFs problem. *Risk:* near zero. Nothing is written, nothing is replaced.

Phase 1 — Upload with validation (3 weeks)

The scheduler uploads a workbook; the system validates, shows a diff, publishes on approval. SSO and roles land here.

Delivers: the class of errors in §1.2 becomes impossible to publish. *Also lands here:* version-checked concurrent edits (§8.8), and traceability from every meeting back to its source file and hash (§8.9).

Phase 2 — Room booking (6 weeks, was 4)

Requests, approval routing, the availability finder, the exclusion constraint.

The increase is Part 8, and it is not padding. Precedence and eviction with acknowledgement (§8.1), deputies and 48-hour escalation (§8.3), auto-refuse at five days, recurring bookings clamped to term end (§8.6), and the first background job runner — which nothing before this phase needed.

Delivers: booking stops being an email thread. *This is the phase where the institution starts depending on the system,* which is why Part 11's ownership question must be answered before it starts, not after.

Phase 3 — Change notification (2 weeks)

Generated diff paragraphs, email and web push to affected cohorts. Separate from the ICS path, for the reason in §2.3.

Phase 4 — Utilisation reporting (2 weeks, frequency only)

Frequency rate ships here. Occupancy and utilisation do not, because they need real enrolment per meeting and publishing them on cohort headcount would report every elective and every split tutorial as a near-empty room (§5.5).

Phase 5 — Course pages and add/drop (a term of work, decided separately)

Only once the schedule is trustworthy. The standard pattern is well established: a shopping cart students fill before registration opens, a validation pass checking prerequisites, credit limits, time conflicts and consent, then enrolment with waitlists where a section is full. This needs a decision about whether it integrates with the existing ERP or replaces part of it, which is Part 11.

Part 11 — Decisions management must make

These block design, not implementation. Each needs an owner and an answer.

1. Where does it run, and who owns it after I graduate? The requirement is campus-local, which rules out the public cloud. It needs a maintainer named before go-live. A system nobody can maintain in two years is worse than the spreadsheet.
2. Does it become the system of record? If the office keeps editing workbooks and the system imports them, it is a publisher and the workbook stays authoritative. If the office edits in the system, it is the record and the workbook becomes an export. Phase 0 and 1 work either way; Phase 2 onward assumes the second.
3. Who approves a booking, per room, and who deputises? Approval routing is a policy question, not a technical one, and it cannot be guessed. Every room needs a named approver *and* a named deputy, or requests rot whenever someone takes leave (§8.3).
- 3b. Does teaching evict an approved booking? The precedence order in §5.3 says yes, with acknowledgement and notification. The alternative is that a society booking can block the timetable being published. Management should confirm this explicitly, because the first time it happens someone will be unhappy and the answer needs to have been decided in advance rather than in the moment.
- 3c. May students request rooms? The permission exists and is switched off in v1 (Appendix I). Turning it on is a policy call about who may commit institutional space.
7. Do we use Exchange room mailboxes as the booking substrate? This is the largest open architectural question and it belongs to IT, not to me (§2.3). It decides whether Phase 2 builds a booking system or integrates with one, and it materially changes who has to maintain what after I graduate.
4. What integrates? Blackboard, the ERP, Outlook room resources. Each is a separate piece of work and none is required for Phase 0 to 3.
5. Who may see what? Are schedules public, campus-only, or role-scoped? This determines whether Phase 0 needs authentication at all.
6. Data ownership and exit. The institution owns the data and can export it in full at any time. Worth stating explicitly in the proposal rather than leaving implied.

Part 12 — What already exists, and how to check it

This is the one part of the document a reader cannot verify from the evidence in it, because it is a claim about my own work. So every line below is stated as something that can be run or opened, and the limits are stated with the capabilities.

Repository: github.com/EchoRover/links.

12.1 Working, and checkable

Claim	Check it
Reads all twelve cohorts from the office workbooks, 269 blocks, taking group membership from merged cell spans	<code>python3 tools/read_xlsx.py</code>
Year 3 CSE comes out an exact match against a hand-verified week on day, time, course, room, kind and group	diff in §12.3
Produces the room-wise pivot the office asked for: 160 bookings across 19 rooms, shared classes merged	<code>python3 tools/room_schedule.py</code>
Finds the clash, the naming conflicts and the impossible times in Part 1	same command, printed output

Claim	Check it
Reads the published PDFs where only a PDF exists, and refuses rather than guesses on an unreadable one	<code>python3 tools/parse_sheet.py --all</code>
Shows a parse laid out like the sheet, for eyeballing	<code>python3 tools/review.py</code>
Checks the shipped timetable against the sheet's own credit table	<code>python3 tools/check_timetable.py</code>
A live student-facing site: Sem 5 timetable, free-room finder, campus bus times, 3D floor plans, wall-display board	<code>linkcs.vercel.app</code>
Campus bus data verified against the transport office's own workbook, including a 20:50 departure that breaks the 20-minute cadence	<code>data/schedules/, js/bus-data.js</code>

About 1,800 lines of Python across seven tools, plus the site.

12.2 What this is *not*

Stated because the gap between a working script and institutional software is where proposals like this usually mislead:

- No database. Everything above reads files and prints. There is no schema, no constraint, no transaction — Appendix C is a design, not a running thing.
- No authentication, no roles, no writes. Nothing in Part 5 beyond reading and publishing exists.
- No booking, in any form. All of Part 8 is design.
- The tools are for me, run from a terminal. They are not a product and no member of staff could use them as they stand.
- The PDF reader does not cover everything. Four Year 1 sheets are untagged and have no exact reading; two MTech sheets are tagged but yield zero blocks and are unhandled. The workbooks make this moot, but only while the workbooks keep arriving.

Honestly: Phase 0 is perhaps 60% done, and Phases 1 to 4 are 0% done. Reading and validating is the part that exists. Storing, authenticating, writing and booking is the part that does not, and it is the larger part.

12.3 The single check that matters most

If one claim in this document deserves testing before any of it is believed, it is that the reader is exact. The test is a diff against a week verified by hand against the printed sheet and independently against that sheet's own credit table:

```
python3 - <<'EOF'
import sys, json; sys.path.insert(0,'tools')
from read_xlsx import all_cohorts
got = {(b['day'],b['start'],b['end'],b['course'],b['room'],b['kind'],b['group'])
        for b in all_cohorts()['24A1CSEBSEM5']['blocks']}
tt = json.load(open('data/linkcs/timetable.json'))
want = {(tt['days'][d],b['start'],b['end'],b['course'],b['room'],b['kind'],b['group'])
        for d,bs in tt['week'].items() for b in bs}
print('EXACT MATCH' if got == want else f'DIFFERS\n {want-got}\n {got-want}')
EOF
```

Run on 11 September 2026 against the 8 September workbook: EXACT MATCH, 23 blocks, nothing missing and nothing extra — including the five group assignments the sheet never states in words.

Appendix A — Sources

- CollegeNET 25Live — collegenet.com/scheduling/25live

- UniTime, Apereo Foundation — apereo.org/programs/software/unitime, github.com/UniTime/unitime
- RFC 5545 iCalendar · RFC 4791 CalDAV · RFC 6638 CalDAV scheduling · RFC 7986 REFRESH-INTERVAL
- RFC 8984 JSCalendar, and draft-ietf-calext-jscalendar-icalendar — the two formats are expected to coexist; emit iCalendar
- Registration UX patterns: Johns Hopkins SIS, Michigan State SIS, Case Western SIS, Tufts SIS public documentation
- Space utilisation measures: standard HE frequency/occupancy/utilisation definitions
- Brand values read from iitdabudhabi.ac.ae/assets/css/new-style.css

Appendix B — Reproducing the findings

<code>python3 tools/read_xlsx.py</code>	every cohort, from the workbooks
<code>python3 tools/room_schedule.py</code>	the room pivot, clashes, naming conflicts
<code>python3 tools/parse_sheet.py --all</code>	the published PDFs
<code>python3 tools/review.py</code>	a parse, laid out like the sheet

Technical appendices

Everything below is implementation detail. Part 1 to 10 can be read without it.

Appendix C — Schema

Postgres. Written out because the constraints are the design, not an afterthought.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

```
-- ----- estate -----
```

```
CREATE TABLE building (
  code      text PRIMARY KEY,          -- 'M4'
  name      text NOT NULL,
  name_ar   text
);
```

```
CREATE TABLE room (
  code      text PRIMARY KEY,          -- 'M4-0-019'. THE identity.
  deputy_id bigint REFERENCES person(id), -- so a request cannot rot on leave
  building  text NOT NULL REFERENCES building(code),
  floor     text NOT NULL,             -- 'G', '1F'
  plate_en  text,                      -- what the door says
  plate_ar  text,
  capacity  integer,
  kind      text NOT NULL              -- classroom | lab | hall | seminar
      CHECK (kind IN ('classroom', 'lab', 'hall', 'seminar', 'other')),
  bookable  boolean NOT NULL DEFAULT true,
  approver_id bigint REFERENCES person(id),
  plate_photo text,                   -- evidence beats every document
  verified_on date
);
```

```
-- A name a SOURCE claims for a room. Kept so disagreements stay visible
-- instead of one source silently winning.
```

```
CREATE TABLE room_name_claim (
  room_code text NOT NULL REFERENCES room(code),
  source    text NOT NULL,             -- '24A1CSEBSEM5', 'door plate'
  claimed   text NOT NULL,
  seen_on   date NOT NULL,
  PRIMARY KEY (room_code, source, claimed)
);
```

```
-- ----- calendar -----
```

```
CREATE TABLE term (
  id      text PRIMARY KEY,          -- '2026-27-sem1'
  label   text NOT NULL,
  starts  date NOT NULL,
  ends    date NOT NULL,
  CHECK (ends > starts)
);
```

```
CREATE TABLE no_class_day (
```

```

term_id    text NOT NULL REFERENCES term(id),
day        date NOT NULL,
reason     text NOT NULL,
PRIMARY KEY (term_id, day)
);

-- ----- curriculum -----

CREATE TABLE course (
  code      text PRIMARY KEY,           -- 'ACOL351'
  title     text NOT NULL,             -- official, off the sheet
  short     text,                     -- what a pill shows
  dept      text,
  l         integer NOT NULL DEFAULT 0, -- L-T-P-C, stored apart so it
  t         integer NOT NULL DEFAULT 0, -- can be reconciled against
  p         integer NOT NULL DEFAULT 0, -- scheduled minutes
  credits   numeric(4,2) NOT NULL DEFAULT 0
);

CREATE TABLE cohort (
  code      text PRIMARY KEY,           -- '24A1CSEBSEM5'
  label     text NOT NULL,             -- 'Y3 CSE Sem 5'
  term_id   text NOT NULL REFERENCES term(id),
  headcount integer,
  n_groups  integer NOT NULL DEFAULT 2  -- Year 1 runs four
);

CREATE TABLE section (
  id        bigserial PRIMARY KEY,
  term_id   text NOT NULL REFERENCES term(id),
  course_code text NOT NULL REFERENCES course(code),
  cohort_code text NOT NULL REFERENCES cohort(code),
  instructor bigint REFERENCES person(id),

  -- Not every course is taken by the whole cohort. The sheets print
  -- "Applicable if registered" on electives, and a v1 model that assumes
  -- cohort size = attendance produces utilisation figures that are simply
  -- wrong for every elective in the estate.
  enrolment text NOT NULL DEFAULT 'whole-cohort'
           CHECK (enrolment IN ('whole-cohort','elective','shared')),
  registered integer,                -- actual heads, once registration freezes
  UNIQUE (term_id, course_code, cohort_code)
);

-- A course taught once to several cohorts at the same hour in the same room.
-- AHUL256, AHUL261 and AGRL130 are each taught to Y3 CSE and Y3 EEN together,
-- and most of Year 1 is shared across all four branches. Without this, a room
-- pivot reports every shared class as a clash: 248 cohort entries in the
-- current term are 160 real bookings.
CREATE TABLE section_shares (
  section_id bigint NOT NULL REFERENCES section(id),
  cohort_code text NOT NULL REFERENCES cohort(code),
  groups      int[] NOT NULL DEFAULT '{}',
  PRIMARY KEY (section_id, cohort_code)
);

```

```

-- ----- pattern and occurrence -----
-- A weekly class is ONE editable fact ("Wednesdays 15:30, M4-0-019, from
-- 20 Aug to 16 Dec") and SIXTEEN things that occupy a room. Store only the
-- first and the exclusion constraint cannot see overlaps. Store only the
-- second and moving a class means rewriting sixteen rows and losing the fact
-- that they were ever one thing.
--
-- So: both. `meeting` is the editable truth and what the interface edits.
-- `occupancy` is the materialised enforcement surface, regenerated inside the
-- same transaction whenever its meeting changes. This is the RFC 5545 model
-- (RRULE + EXDATE + modified instances) with the expansion persisted so the
-- database can police it.

```

```

CREATE TABLE meeting (
  id          bigserial PRIMARY KEY,
  revision_id bigint NOT NULL REFERENCES revision(id),
  section_id  bigint REFERENCES section(id),
  booking_id  bigint REFERENCES booking(id),
  room_code   text NOT NULL REFERENCES room(code),
  weekday     integer NOT NULL CHECK (weekday BETWEEN 1 AND 7),
  starts      time NOT NULL,
  ends        time NOT NULL,
  tz          text NOT NULL DEFAULT 'Asia/Dubai',    -- never store bare UTC
  from_date   date NOT NULL,
  until_date  date NOT NULL,
  kind        text NOT NULL,
  groups      int[] NOT NULL DEFAULT '{}',          -- {} = whole cohort
  expected    integer,                             -- heads, for utilisation
  CHECK (ends > starts),
  CHECK (until_date >= from_date),
  CHECK (num_nonnulls(section_id, booking_id) = 1)
);

```

```

-- One instance. Generated from a meeting, then editable in its own right so a
-- single week can move or be cancelled without touching the pattern – which is
-- exactly what a room change for one session is.

```

```

CREATE TABLE occupancy (
  id          bigserial PRIMARY KEY,
  meeting_id  bigint NOT NULL REFERENCES meeting(id) ON DELETE CASCADE,
  room_code   text NOT NULL REFERENCES room(code),
  during      tstzrange NOT NULL,
  kind        text NOT NULL
    CHECK (kind IN ('lecture','tut','lab','proj','help',
                    'booking','hold','exam','maintenance')),
  status      text NOT NULL DEFAULT 'confirmed'
    CHECK (status IN ('confirmed','provisional','cancelled')),
  detached    boolean NOT NULL DEFAULT false,      -- edited away from its pattern
  precedence  smallint NOT NULL,                  -- 1 exam .. 5 provisional; see 5.3
  note        text,

  CHECK (upper(during) > lower(during)),

  EXCLUDE USING gist (room_code WITH =, during WITH &&)
  WHERE (status <> 'cancelled')

```

```
);
```

```
-- Pending requests deliberately reserve NOTHING, so they are not in occupancy.
-- A system where asking holds a room gets used to hoard rooms.
```

```
CREATE INDEX ON occupancy USING gist (during);
CREATE INDEX ON occupancy (meeting_id);
CREATE INDEX ON meeting (revision_id);
CREATE INDEX ON meeting (section_id);
```

```
-- ----- bookings -----
```

```
CREATE TABLE booking (
  id          bigserial PRIMARY KEY,
  requester_id bigint NOT NULL REFERENCES person(id),
  purpose     text NOT NULL,
  attendees   integer,
  status      text NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'approved', 'refused',
                                'withdrawn', 'cancelled')),
  decided_by  bigint REFERENCES person(id),
  decided_at  timestampz,
  reason      text, -- required on refusal
  created_at  timestampz NOT NULL DEFAULT now()
);
```

```
-- ----- versioning -----
```

```
CREATE TABLE revision (
  id          bigserial PRIMARY KEY,
  term_id     text NOT NULL REFERENCES term(id),
  n           integer NOT NULL, -- 1, 2, 3...
  source      text, -- uploaded filename
  source_hash text, -- sha256 of the workbook
  stamp       text, -- the sheet's own footer stamp
  author_id   bigint NOT NULL REFERENCES person(id),
  published_at timestampz,
  summary     text, -- the generated diff paragraph
  UNIQUE (term_id, n)
);
```

```
CREATE TABLE person (
  id          bigserial PRIMARY KEY,
  sso_subject text UNIQUE NOT NULL, -- from institutional SSO
  name        text NOT NULL,
  email       text NOT NULL,
  role        text NOT NULL
              CHECK (role IN ('student', 'faculty', 'approver', 'scheduler', 'admin')),
  cohort_code text REFERENCES cohort(code),
  active      boolean NOT NULL DEFAULT true
);
```

```
CREATE TABLE audit (
  id          bigserial PRIMARY KEY,
  at          timestampz NOT NULL DEFAULT now(),
```

```

actor_id    bigint REFERENCES person(id),
action      text NOT NULL,
entity      text NOT NULL,
entity_id   text NOT NULL,
before      jsonb,
after       jsonb
);

```

Regenerating occurrences is not a background job. A meeting edit deletes and re-inserts its occurrences *inside the same transaction*, so the exclusion constraint refuses the edit if any generated instance would collide. Do it asynchronously and there is a window where the database holds a double-booking. The one exception is an occurrence marked detached: it was deliberately moved away from its pattern and regeneration must leave it alone.

Why **meeting** carries **revision_id**. Publishing a revision writes a new set of rows rather than mutating the old ones. "What did the timetable say on 8 September" is then a query, and a bad publish is reverted by pointing the term at the previous revision rather than by restoring a backup.

Appendix D — The validation catalogue

Every rule an upload must pass. The numbering is the error code the interface shows.

Structural (the file itself)

Code	Rule	Caught in the real data?
S1	Every sheet has a recognisable day column	—
S2	Every day band has group labels in column B	—
S3	A parsed sheet yields more than zero blocks	Yes. Both MTech sheets yielded zero and an earlier checker called that "ok"
S4	The workbook carries a term the system knows	—

Per block

Code	Rule	Caught?
B1	End time is strictly after start time	Yes. Y1 ELE and Y1 EEN: "15:30 to 14:20"
B2	Times fall inside the institution's teaching day (07:00–21:00)	—
B3	Room code parses and exists in room	—
B4	Course code exists in course	—
B5	The block's groups are a subset of its cohort's groups	—
B6	Duration is a plausible multiple (50 / 80 / 110 / 170 min)	—

Cross-block

Code	Rule	Caught?
X1	No room holds two different courses at overlapping times	Yes. Wed 17:00, M4-0-019
X2	No cohort-group is in two places at once	—
X3	No instructor is in two places at once	—
X4	A course taught to several cohorts at one time and place is ONE entry	Yes. 248 cohort entries collapse to 160 bookings; without this, 87 false clashes

Curriculum

Code	Rule	Caught?
C1	Every course on the grid appears in the L-T-P-C table	—
C2	Every course in the L-T-P-C table appears on the grid	—
C3	Scheduled contact hours reconcile against L-T-P-C, per group	Yes. This is what proves group membership
C4	Declared exceptions are listed, not inferred	AC0D310 is 0-0-6-3 with one slot reserved
C5	Non-contact blocks are excluded from C3	Yes. “Help Session if required” broke the reconciliation until excluded

Naming

Code	Rule	Caught?
N1	A room code maps to one plate name across all sheets	Yes. Six rooms fail this
N2	A plate name maps to one room code	Yes. “Classroom 8” is both M4-0-019 and M4-1-011
N3	Where a photographed plate exists, it wins, and disagreement is reported	Yes. Y1 CHE calls M4-0-011 “Lecture Hall”; the plate says Classroom 3

Severity matters. S and B failures *reject* the upload. X and C failures reject unless the scheduler explicitly acknowledges each one, because a genuine timetable can carry a deliberate overlap. N failures are *warnings* that publish: a naming disagreement should be visible, not blocking.

Appendix E — API surface

REST, JSON, thin. Public reads need no auth; everything else is SSO-gated.

Public reads

```

GET /api/terms                the terms, current first
GET /api/cohorts              every cohort, with labels
GET /api/cohorts/{code}/week?revision=  meetings, grouped by day
GET /api/rooms                every room, plate names, capacity
GET /api/rooms/{code}/week?from=  occupancy for a room
GET /api/rooms/free?day=&from=&to=&seats=  the availability finder
GET /api/instructors/{id}/week
GET /ics/cohort/{code}.ics      RFC 5545 feed
GET /ics/room/{code}.ics
GET /ics/instructor/{id}.ics
GET /api/revisions?term=      revision history with diff summaries
GET /api/revisions/{n}/diff    sets in, out, moved

```

Authenticated

```

POST /api/bookings            create a request
    {room_code, during, purpose, attendees}
GET /api/bookings?status=&mine=
POST /api/bookings/{id}/approve
POST /api/bookings/{id}/refuse    {reason} -- required
POST /api/bookings/{id}/withdraw

POST /api/terms/{id}/uploads      multipart, the workbook
    -> 202 {upload_id}
GET /api/uploads/{id}              {status, findings[], diff}
POST /api/uploads/{id}/publish     {acknowledge: [...]}

GET /api/reports/utilisation?term=&from=&to=
GET /api/reports/naming-conflicts
GET /api/export/term/{id}.xlsx    full export, any time

```

Response shape, `/api/cohorts/{code}/week`

```

{
  "cohort": {"code": "24A1CSEBSEM5", "label": "Y3 CSE Sem 5", "n_groups": 2},
  "term":   {"id": "2026-27-sem1", "starts": "2026-08-20", "ends": "2026-12-16"},
  "revision": {"n": 6, "published_at": "2026-09-08T09:30:00Z",
               "stamp": "8th September 2026 - 1:30pm"},
  "meetings": [
    {"day": "Wednesday", "start": "15:30", "end": "16:20",
      "course": {"code": "ACOL351", "short": "Algos"},
      "room": {"code": "M4-0-019", "plate": "Classroom 5",
               "plate_disputed": true, "also_called": ["Classroom 3", "Classroom 8"]},
      "kind": "tut", "groups": [], "status": "confirmed"}
  ]
}

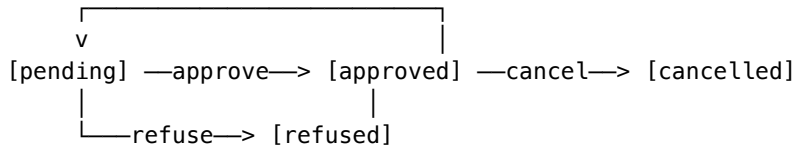
```

`plate_disputed` is not decoration. It is the interface's licence to show the reader that the documents disagree, instead of picking one and being confidently wrong.

Appendix F — State machines

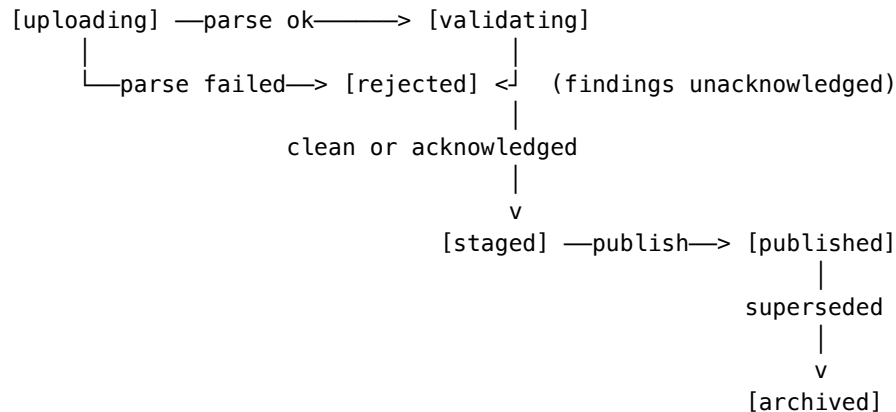
Booking

withdraw



- approve writes an occupancy row inside the same transaction. If the exclusion constraint fires, the approval fails and the approver is told which booking holds the slot.
- refuse requires a reason. A refusal with no reason generates a follow-up email to the approver anyway, so the field may as well be mandatory.
- cancelled keeps the row. The slot is released by the partial index, and the history survives.

Revision



Only one revision per term is published. Rolling back is publishing an older revision, which creates a new revision row recording that it happened.

Appendix G — Screen specifications

G1 — Cohort week (the landing view)

- Layout. Days as rows, time 08:00–19:00 across. Matches the printed sheet’s orientation, which is the artefact everyone already reads.
- Block. Course code in mono at 11px; kind badge; group badge only when the block does not apply to the whole cohort; room plate; time. When a block is narrow, the badges survive and the room is what truncates — the badges are what the reader is checking.
- Overlap. Blocks that overlap stack into lanes. Nothing is ever hidden behind anything else.
- State. provisional draws a dashed outline plus a marker; colour alone never carries it.
- Now. A hairline at the current time on today’s row, and only on today’s row.
- Empty. A term with no published revision says so and links to the upload screen if the viewer is a scheduler. It never renders an empty grid.

G2 — Room day (the booking surface)

- One room, one day, time across the full width.
- Teaching in solid fill, approved bookings in a lighter fill of the same hue, pending requests hatched.
- Free time is genuinely empty and is the click target. Drag across it to open a request with times pre-filled.
- A right rail shows the room: plate name, capacity, kind, approver, and — if the plate is disputed — every name the documents give it.

G3 — Room week grid (the office view)

- Rooms down the side, weekdays across, each cell a compressed strip.

- This is the screen where a clash is visible without anyone searching for one.
- Sort by building, then floor, then code. Not alphabetically: physical adjacency is what the reader is thinking about.

G4 — Availability finder

I need a room [90] minutes on [Tuesday] after [14:00]
for [40] people in [any building ▼] [Find]

Results ranked by capacity fit, closest first, not alphabetically. Putting a seminar of twelve into a 120-seat lecture hall is a real cost, and a list sorted by room code invites exactly that.

G5 — Utilisation

- Rooms against hours, cell colour by frequency rate.
- A sequential ramp, not a rainbow. Dark equals busy.
- Below it, the three headline numbers per building: frequency, occupancy, utilisation.
- A “never used” list, which is the finding management acts on.

G6 — Mobile

Single column. The top of the screen answers one question: what is next, where, and how long until it starts. Everything else is below the fold and that is correct — the phone case is a person walking between buildings.

Appendix H — Emitting iCalendar

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//IITD Abu Dhabi//One Timetable//EN
CALSCALE:GREGORIAN
METHOD:PUBLISH
X-WR-CALNAME:Y3 CSE Sem 5
REFRESH-INTERVAL;VALUE=DURATION:PT1H
X-PUBLISHED-TTL:PT1H
BEGIN:VEVENT
UID:meet-8842@timetable.iitdabudhabi.ac.ae
DTSTAMP:20260908T093000Z
DTSTART;TZID=Asia/Dubai:20260909T153000
DTEND;TZID=Asia/Dubai:20260909T162000
RRULE:FREQ=WEEKLY;BYDAY=WE;UNTIL=20261216T235900Z
EXDATE;TZID=Asia/Dubai:20261028T153000
SUMMARY:ACOL351 Tutorial
LOCATION:M4-0-019 (Classroom 5)
DESCRIPTION:Analysis and Design of Algorithms\nTutorial\nAll groups
CATEGORIES:TUTORIAL
SEQUENCE:3
END:VEVENT
END:VCALENDAR
```

Points that are easy to get wrong:

- **UID** is stable for the life of the meeting. Regenerating it on every publish makes every subscriber’s calendar duplicate the entire term.
- **SEQUENCE** increments on change, so clients update rather than duplicate.
- Store local time with **TZID**, never bare UTC. 15:30 Asia/Dubai and its UTC instant are different facts, and if the term shifts the local time is what was meant. This is the specific mistake JSCalendar (RFC 8984) was designed to fix.

- No-class days become **EXDATE**, not deleted occurrences.
- Set **REFRESH-INTERVAL** and **X-PUBLISHED-TTL** anyway. They cost nothing and the clients that honour them are the ones that poll fast. Google ignores both.

Appendix I — Permissions

Action	Viewer	Student	Faculty	Approver	Scheduler	Admin
See published schedules	•	•	•	•	•	•
Subscribe to a feed	•	•	•	•	•	•
See room availability	•	•	•	•	•	•
Request a room			•	•	•	•
See who requested				•	•	•
Approve or refuse				• ¹	•	•
Upload a term					•	•
Publish a revision					•	•
Roll back a revision						•
Edit rooms and people						•
Read the audit log					•	•

* Only for rooms where they are the named approver.

Students cannot request rooms in v1. That is a policy question for Part 11, not a technical limit — the row exists, the permission is simply off.

Appendix J — Testing

Scale here is tiny: eleven cohorts, about 300 meetings, nineteen rooms. Load is not the risk. Correctness is. The test effort goes almost entirely into the validator and the importer.

Golden-file tests. The Sem 5 CSE week is hand-verified against the sheet and against its own credit table. It is the fixture: import it, assert 23 meetings exactly, on day, time, course, room, kind and group. Any importer change that breaks it is wrong.

Property tests. For any generated set of meetings, publishing then exporting then re-importing returns the same set. Round-tripping is where importers rot.

Constraint tests. Two concurrent approvals for the same slot: exactly one succeeds. Run it against a real Postgres, not a mock — the guarantee is the database's, so a mock tests nothing.

Regression corpus. Every defective sheet found this week becomes a fixture: the “15:30 to 14:20” file, the zero-block MTech sheets, the M4-0-019 clash, the “Classroom 8” mislabel. Each asserts the specific code from Appendix D.

What not to test. Rendering. Screenshot tests on a timetable grid are famously brittle and catch almost nothing that matters here.

Appendix K — Effort

Assuming one developer, part-time alongside coursework. Ranges, not promises.

Phase	Work	Estimate
0	Schema, importer, discover mode, public read views, ICS	3 weeks
1	SSO, roles, upload with validation, diff, publish, versioned edits	3 weeks
2	Bookings, precedence and eviction, deputies, escalation, job runner	6 weeks
3	Change notification, generated diff paragraphs, mail path	2 weeks
4	Frequency-rate reporting (occupancy needs enrolment)	2 weeks
5	Course pages, add/drop	a term, scoped separately

Sixteen weeks to a system that publishes, validates and books, at part-time pace. Drafts 2 and 3 moved this up from thirteen, and the increase is Part 8 — the failure modes that were missing from the first plan, not scope that was added to it.

Phase 0 is short because much of it exists. The importer that reads the office workbooks — including group membership from merged cell spans — is written and verified. The room pivot is written. A student-facing site carrying the Sem 5 timetable, a free-room finder and a wall-display board is live.

The honest risk is not the code. It is Part 11, question 1: who owns this after I graduate. Every estimate above assumes that gets answered before Phase 2, because Phase 2 is where the institution starts depending on it.